SMU Lyle School of Engineering

Reinforcement Learning

Deep Reinforcement Learning

Based on An Introduction to Deep Reinforcement Learning**
+ Sutton/Barto* Sections 9.7 and 15.7

Michael Hahsler, SMU

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018

**Vincent François-Lavet, Peter Henderson, Riashat Islam, Marc G. Bellemare and Joelle Pineau (2018), "An Introduction to Deep Reinforcement Learning", *Foundations and Trends in Machine Learning*: Vol. 11, No. 3-4. DOI: 10.1561/2200000071.

Online Material

Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- Part I: Tabular Methods
  - Dynamic programming
  - Monte Carlo methods
  - Temporal-difference learning
  - Multi-step bootstrapping
  - Planning and learning with tabular methods
- Part II: Approximate Solution Methods
  - Prediction and Control using Approximation
  - Eligibility Traces
  - Policy Gradient Methods
- **Part III: Modern RL Methods**
  - **Deep Reinforcement Learning**
  - Current Applications

Summary of Notation*

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or v_*

Temporal Difference Learning

U_t	target for estimate at time t
δ_t	TD error

*Some authors use a slightly different notation.

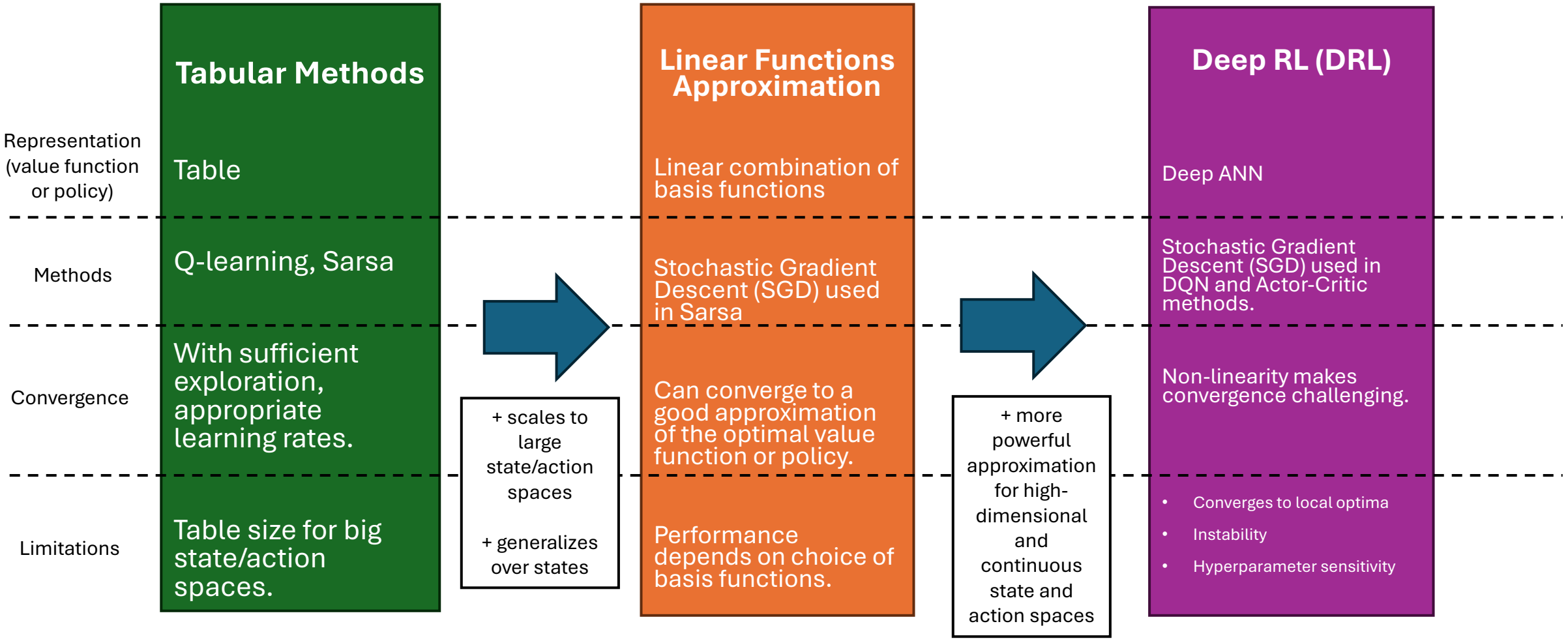
MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a.
$p(s' \mid s, a)$	probability of transition to state s' fom state s taking action a.
$r(s, a)$	expected immediate reward from state s after action a.
$r(s, a, s')$	expected immediate reward from state s to s' with action a.
$\pi(a \mid s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Approximation

$\mu(s)$	on policy distribution (fraction of time spent in state)
α	step size (gradient descent learning rate)
$\hat{v}(s, \boldsymbol{w})$	approximate value function
$\hat{q}(s, a, \boldsymbol{w})$	approximate action-value function
$\pi(a \mid s, \boldsymbol{\theta})$	approximate policy
$\widehat{VE}$	mean squared value error
$\nabla f(x)$	gradient of function f evaluated at x

From Tabular RL to Deep RL



Disclaimer: Convergence

- So far, we have had strong convergence guarantees:
  - **Tabular RL** has the contraction mapping guarantee (Bellman Operator): guaranteed convergence to the optimal value function (a fixed point). The policy improvement theorem shows that the policy improves in every step.
  - **TD/Q-learning** converges with sufficient exploration and diminishing learning rates.
  - **Linear approximation:** Mean squared value error creates a convex optimization problem. On-policy learning converges to the global optimum.
    But off-Policy convergence is not guaranteed: Off-policy + bootstrapping + function approximation = **deadly triad**

- Non-linear approximation (DRL) breaks most guarantees in RL!
  - **Loss of contraction mapping** due to a non-linear projection. A fixed point may not exist or be reachable.
  - **Monotonic policy improvement is lost:** Policy can get worse.
  - **Stochastic approximation guarantees break**: Targets depend on parameters (bootstrapping), and **gradients are biased.**

Value-based DRL Methods

Use ANNs to represent a parameterized value function that is updated via gradient descent.

Remember: Value Functions

- Immediate reward: $r_t = r(s_t, a_t, s_{t+1})$
- Value functions:
  - $V^\pi(s) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s, \pi]$
  - $Q^\pi(s,a) = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid S_t = s, A_t = a, \pi]$

The expectation reflects all uncertainty:
- Stochastic policy
- Stochastic transitions
- Stochastic reward model

- Bellman equation:

$$Q^\pi(s,a) = \sum_{s' \in \mathcal{S}} p(s,a,s') \left(r(s,a,s') + \gamma Q^\pi\left(s', a = \pi(s')\right) \right)$$

- Optimality condition:

$$V^*(s) = \max_{\pi \in \Pi} V^\pi(s) \quad \text{and} \quad Q^*(s,a) = \max_{\pi \in \Pi} Q^\pi(s,a)$$

- Optimal policy: $\pi^*(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q^*(s,a)$

Remember: Tabular Q-Learning

- An extremely influential method developed by Watkins (1989).
- Tabular update rule only uses Sars:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{target}} - \underbrace{Q(S_t, A_t)}_{\text{prediction}}]$$

$$\text{TD-error}$$

- Q directly approximates q_* independent of the behavior policy being followed!
  → Off-policy
- Converges to the optimal value function if
  - State-action pairs are represented discretely (in a table).
  - All action are repeatedly sampled in all states (behavior policy keeps exploring).

- **Issue**: $Q(s, a)$ can often not be represented in a table due to too many action-value pairs. Use a parameterized approximation $Q(s, a; \boldsymbol{\theta})$. Deadly triade:
  - Bootstrapping
  - off-policy learning
  - function approximation
- Linear approximation converges for special cases. Non-linear approximation is an issue!

Fitted Q-Learning

Notation: DRL literature uses often Q where our textbook uses $\hat{q}$

- Use a parameterized approximation $Q(s, a; \boldsymbol{\theta})$
- Initial setting for $\boldsymbol{\theta}_0$ such that all Q-value are close to 0 (helps with learning).
- Bootstrapped target value at iteration k:

$$U_k = r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}_k)$$

Q-learning assumes the best action will be taken

- Update:

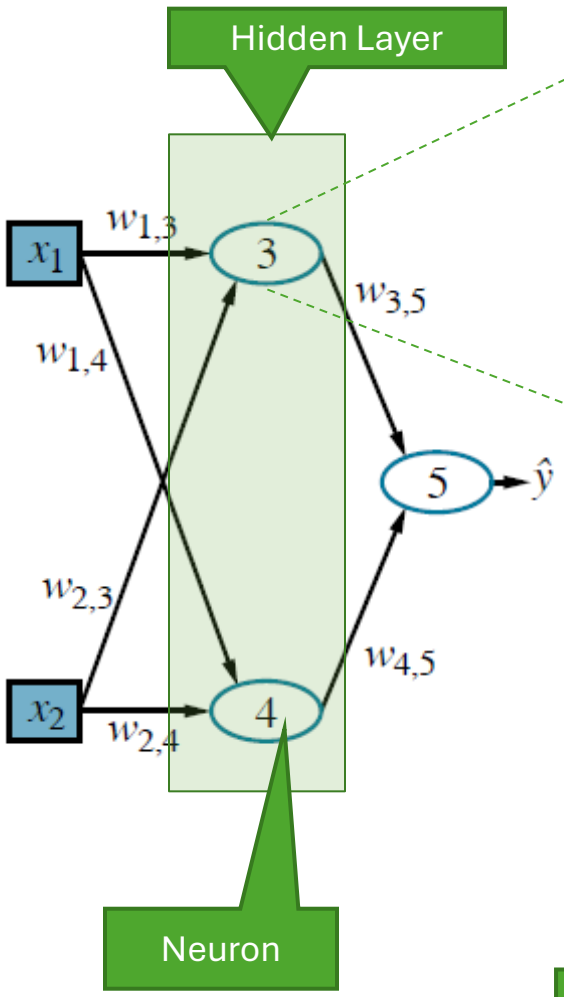
$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha\big(U_k - Q(s, a, \boldsymbol{\theta}_k)\big)\nabla Q(s, a, \boldsymbol{\theta}_k)$$

- Linear approximation with transformed state features can be used.
- Next, we will consider an artificial neural network (ANN) as the approximator.

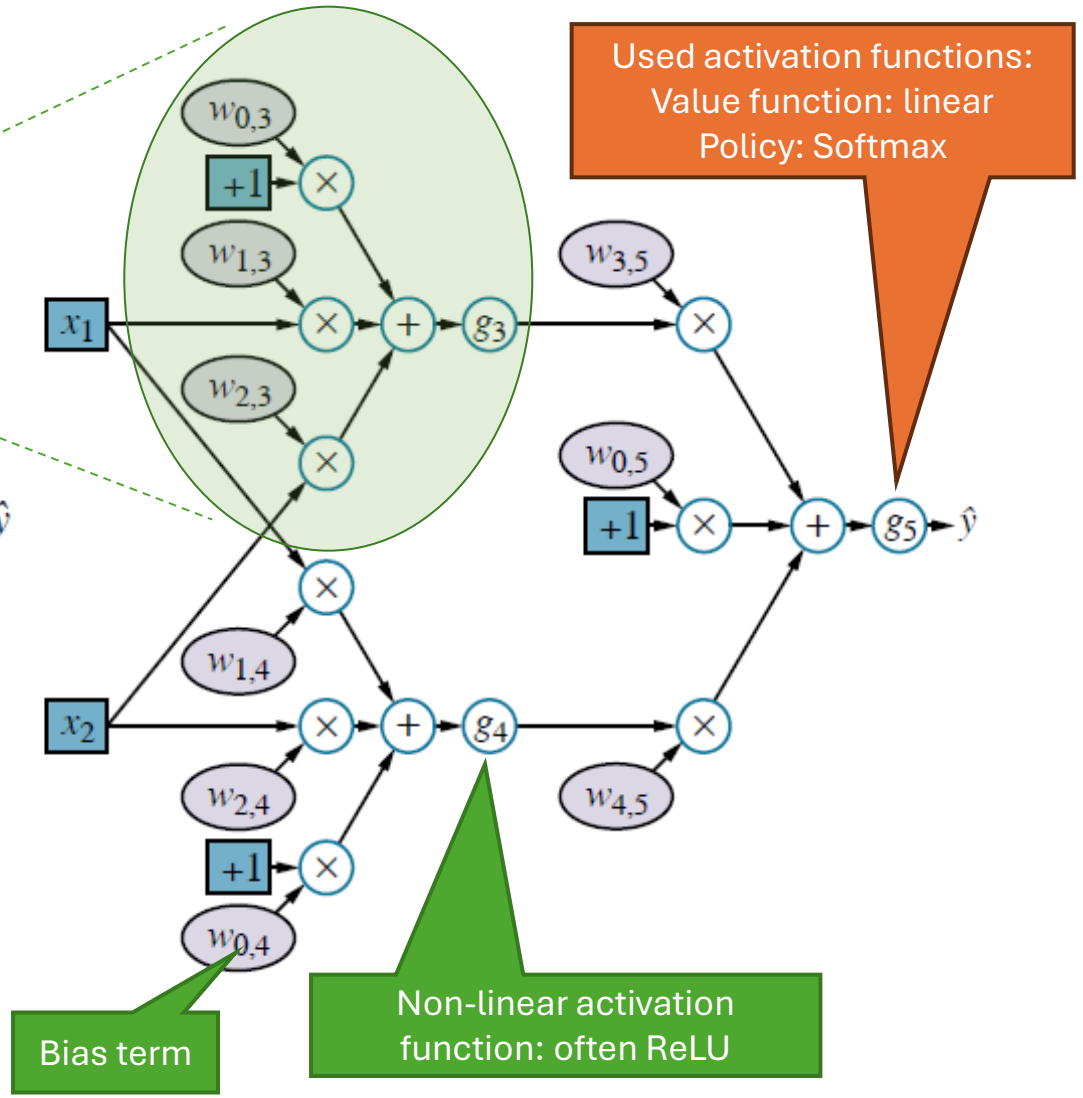
Reminder: Artificial Neural Networks

Superscript $[n]$ means the layer. Layer weights are collected in a matrix.

Network Topology



Computational graph

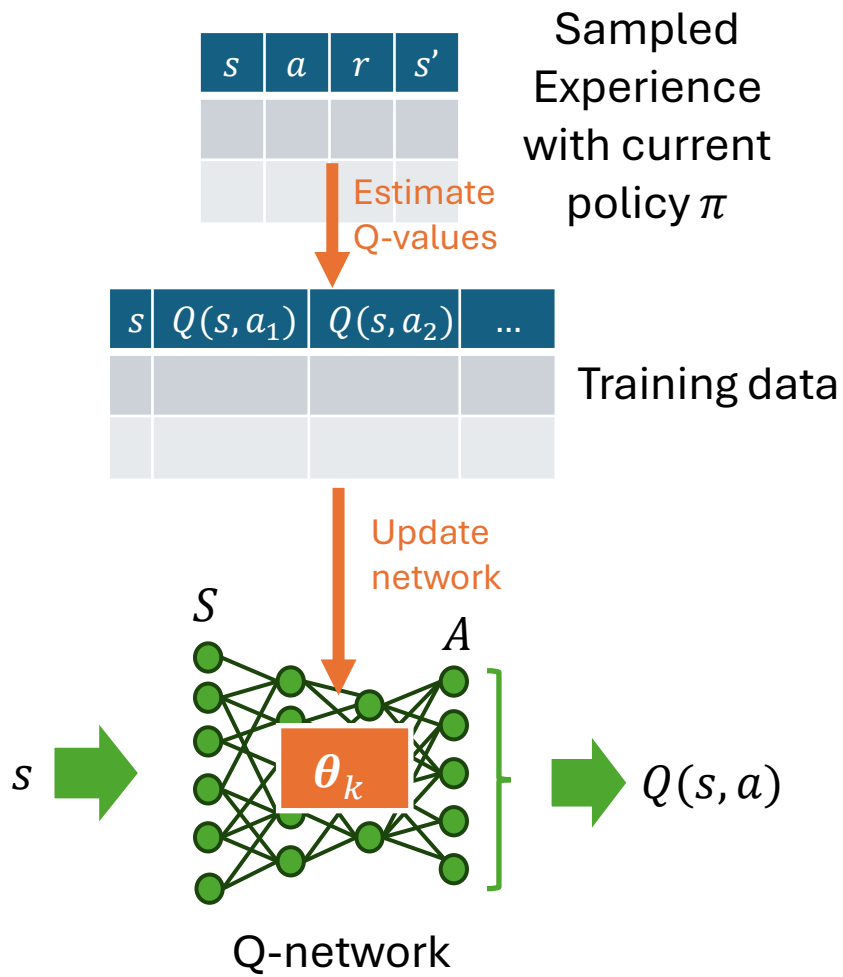


- Represent $\hat{y} = h(\boldsymbol{x}) = g^{[2]}\left(\boldsymbol{W}^{[2]}\, g^{[1]}\left(\boldsymbol{W}^{[1]}\boldsymbol{x}\right)\right)$ as a network of weighted sums with non-linear **activation functions** $g(\cdot)$ (e.g., sigmoid, ReLU).
- Learn weight matrices $\boldsymbol{W}$ from examples using gradient descent with **backpropagation** of prediction errors $L(\hat{y}, y)$.
- ANNs are **universal approximators**. Large networks can approximate any function (has no bias). **Regularization** is typically needed to avoid overfitting.
- **End-to-end learning**: The hidden layer performs "automatic feature engineering.
- **Deep learning** adds more hidden layers and layer types (e.g., convolution layers) for more efficient learning and transfer learning.

Notation: We use for the network weights $\boldsymbol{W}$ often $\boldsymbol{\theta}$

Neural fitted Q-learning (NFQ; Riedmiller, 2005)

- Use an ANN called the **Q-network** as the approximator for the Q-function.
- **Issue**: Online learning does not work well with our hardware. (GPUs only efficiently perform **many** matrix operations in parallel)
- **Solution**:
  - Learn the Q-network offline: **batch learning** with **experience replay** from a sample set of experience containing (s, a, r, s') tuples.
  - Needs off-policy learning: Use **supervised learning (regression)** with a sample from a replay buffer as the training set.
  - The regression target (Q-values) are estimated from the tuples using **temporal differencing**: $Q(s,a) = r + \gamma \max_{a'} Q(s', a')$
- **More issues**:
  - May not converge or become unstable (uses non-linear approximation where the target moves).
  - Q-values tend to be overestimated due to the max operator in the target.



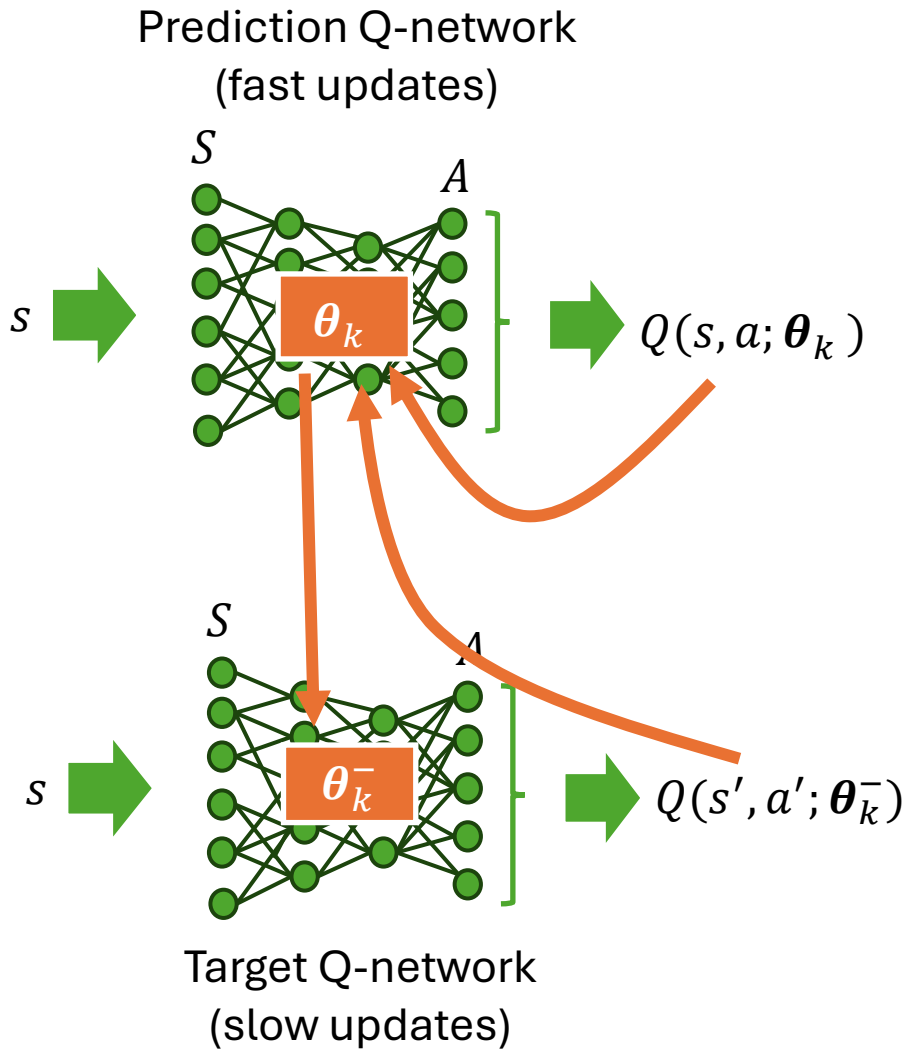
Deep Q-Network (DQN; Minh et al, 2015)

- DNQ addresses the **instability** of NFQ.
- **Idea**: reduce instability by calculating the target from a second Q-network that is updated slowly:

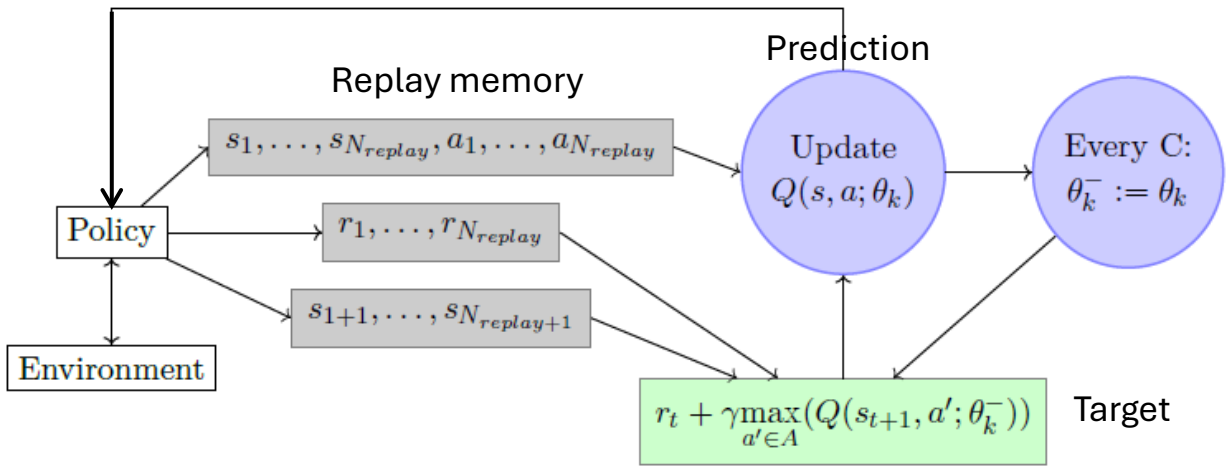
$$U_k = r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}_k^-)$$

- **Updates**:
  - $\boldsymbol{\theta}_k$ is updated every iteration:
    $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha\big(U_k - Q(s, a, \boldsymbol{\theta}_k)\big)\nabla Q(s, a, \boldsymbol{\theta}_k)$
  - $\boldsymbol{\theta}_k^-$ is updated only every C iterations by $\boldsymbol{\theta}_k^- = \boldsymbol{\theta}_k$.

- **Effect**: Target U_k estimates are kept constant for C iterations, instabilities cannot propagate quickly.



DQN is Not a Truly Online Algorithm



1. Initialize $\boldsymbol{\theta}$ and $\boldsymbol{\theta}^-$ so that all $Q(s, a; \boldsymbol{\theta})$ are close to 0.
2. Use an ϵ-greedy policy based on the current $\boldsymbol{\theta}$ to collect experience in the form of N_{replay} many (s, a, r, s') tuples in the replay memory. Note: old tuples may be overwritten.
3. Mini-batches (e.g., 32 elements) are taken randomly from the replay memory to update $\boldsymbol{\theta}$.
4. $\boldsymbol{\theta}^-$ is set to $\boldsymbol{\theta}$ every C iterations.
5. Go back to collecting more experience in step 2.

Double DQN (DDQN; Van Hasselt, 2016)

- **Issue**: Q-learning and DQN use one function in the target to choose the action and evaluate its value. The max creates an **overestimation bias**.

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha \Big(\overbrace{r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}_k)}^{\text{target}} - \overbrace{Q(s, a, \boldsymbol{\theta}_k)}^{\text{prediction}} \Big) \nabla Q(s, a, \boldsymbol{\theta}_k)$$

TD-error

- DDQN **decoupling the action selection and evaluation** by using different parameters:

$$U_k = r + \gamma Q\left(s', \operatorname*{argmax}_a Q(s', a'; \boldsymbol{\theta}_k); \boldsymbol{\theta}_k^-\right)$$

Action evaluation uses more stable $\boldsymbol{\theta}_k^-$

Action selection uses $\boldsymbol{\theta}_k$

New: Advantage Function

- The “advantage” of choosing action a over the baseline given by the state value is

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- Positive advantage means action a is better than the action that $\pi(s)$ chooses.

  For the optimal π^*:

$$A^{\pi^*}(s,a) = \begin{cases} 0 & \text{if } s = s^* \\ < 0 & \text{otherwise.} \end{cases}$$

- **Benefit**: Advantage typically produces a stronger signal to improve the policy.

- V, Q and A can be estimated using Monte Carlo methods (unbiased).
- V, Q and A can be used to calculate a TD-error.
- V, Q and A can be used for parameter updates.

Dueling Network Architecture (Wang, 2015)

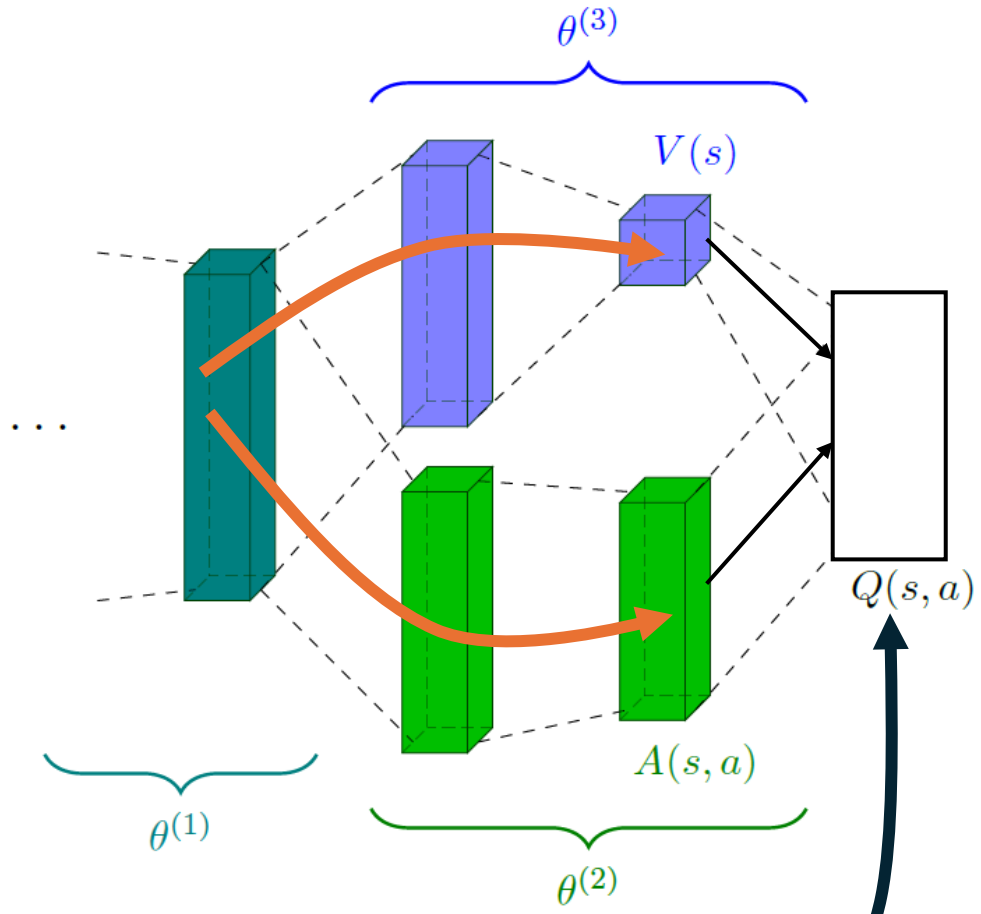
- Dueling Network Architecture only changes the ANN architecture. It learns two components:
  - Value function $V(s; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(3)})$
  - Advantage function $A(s, a'; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(3)})$
- $Q(s, a)$ is calculated using the equation below.

- Reminder: Advantage function

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

  gives the "advantage" of choosing action a over the state value.

- This approach typically leads to improved performance.

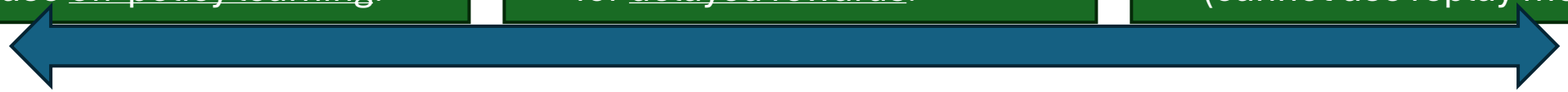


$$Q\left(s, a; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \boldsymbol{\theta}^{(3)}\right) = V\left(s; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(3)}\right) + \left(\mathrm{A}\left(\mathrm{s}, \mathrm{a}; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}\right) - \max_{a' \in \mathcal{A}} \mathrm{A}\left(\mathrm{s}, \mathrm{a}'; \boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}\right)\right)$$

Becomes 0 for $a = a^*$

n-step Methods in DQN-type Algorithms

Bootstrapping (one-step)	**Less Bootstrapping (n-step)**	**MC (no bootstrapping)**
- Instability with function approximation (uses its own prediction). + Can use off-policy learning.	+ Less instability + Quicker information propagation for delayed rewards.	+ Stable - Only on-policy learning leads to sample inefficiency (cannot use replay memory).



- ***n*-step** methods are between Q-learning (one-step) and MC, which does not bootstrap.
- Use n-step target for DQN-type algorithms:

$$U_k^n = \sum_{t=0}^{n-1} \gamma^t r_t + \gamma^n \max_{a' \in \mathcal{A}} Q(s_n, a'; \boldsymbol{\theta}_k)$$

- **Traces** can also be used.

Policy Gradient DRL Methods

Use ANNs to represent and learn a parameterized policy.

Remember:
Policy Gradient Theorem

- We define the performance measure as the true value of following π_θ from the start state s_0

$$J(\boldsymbol{\theta}) \stackrel{\text{def}}{=} v_{\pi_\theta}(s_0) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s_0\right]$$

- The **policy gradient theorem** gives us an analytical expression for the unbiased gradient of the performance with respect to the policy parameters:

$$\nabla J(\boldsymbol{\theta}) = \mathbb{E}_\pi[q_\pi(s,a)\nabla \log \pi(a|s)] \propto \sum_s \mu(s) \sum_a q_\pi(s,a)\nabla\pi(a|s,\boldsymbol{\theta})$$

On-policy distribution under π

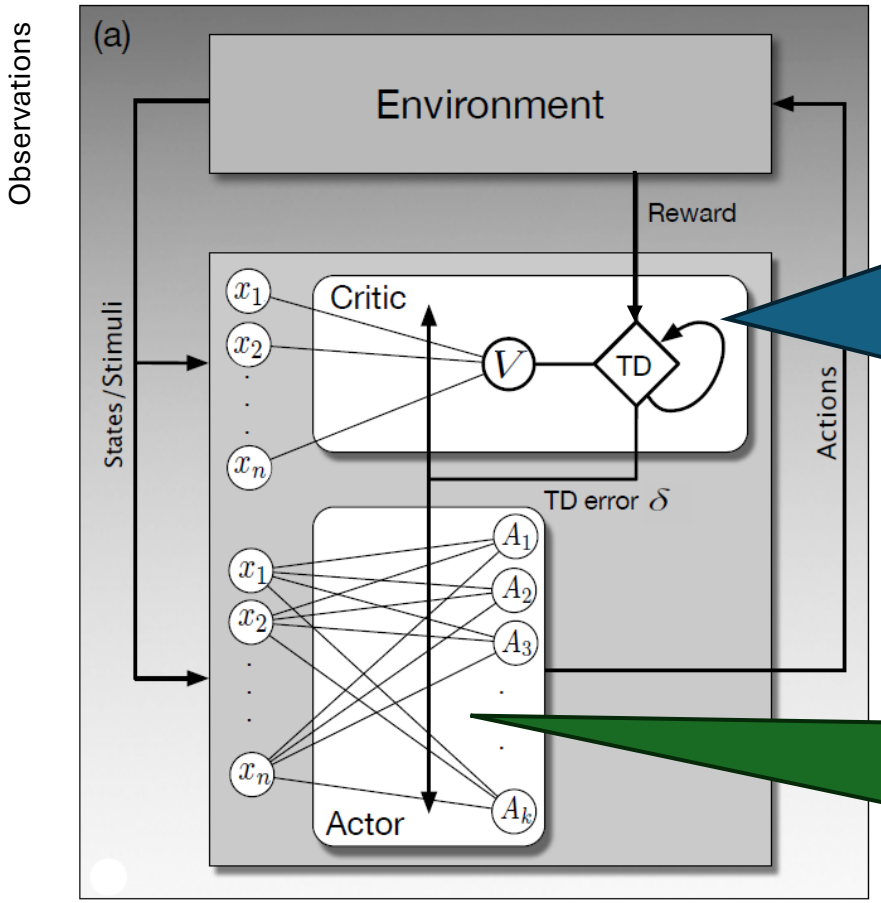
Depends on estimates for q

Depends on the policy approximation function.

- **Convergence guarantee**:
  - Gradient is unbiased and with reducing step size appropriately, ascent **will converge to a local maximum** of $J(\boldsymbol{\theta})$.
  - **Does NOT guarantee: global optimum, fast convergence or stability (with non-linear approximation)**

Neural Actor-Critic Method

Remember: Estimate the value function and a parameterized policy at the same time.



Critic

- Represents value function $Q^{\pi_\theta}(s,a) \approx Q(s,a;\boldsymbol{w})$
- Produces the TD error from the reward signal using its network's state-value prediction.
- Adjusts state-value network using the TD error.
- Does not select actions!

Actor

- Represents policy π_θ
- Selects actions with a policy network.
- Adjusts a policy network based on the TD error
- No access to reward signal!

Section 15.7 in Sutton and Barto (2018)

Neural Actor-Critic Method: One-Step Algorithm

One-step Actor–Critic (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$

Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$

Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

  Initialize S (first state of episode)

  $I \leftarrow 1$

  Loop while S is not terminal (for each time step):

    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$

    Take action A, observe S', R

    $\delta \leftarrow R + \gamma \hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)

    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S,\mathbf{w})$

    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S,\boldsymbol{\theta})$

    $I \leftarrow \gamma I$

    $S \leftarrow S'$

actor

critic

Takes care of discounting γ^t

Policy approximation is soft, so it explores!

TD error: how much is G better than $\hat{v}$

Update the critic (value function) = $TD(0)$

Update the actor (policy) to reduce the TD error

Advantage Actor Critic: A2C

Algorithm 7.2 Advantage Actor-Critic (A2C) Algorithm

1: input: Policy parameters θ, value function parameters w. Learning rates α_θ, α_w, discount factor γ and entropy coefficient β.
2: **for** each iterative step **do**
3: Collect trajectories with transitions $(s_t, a_t, r_t, s_{t+1}, d)$ for $t = 0, 1, \ldots, T$ with $a_t \sim \pi(s_t; \theta)$.
4: **for** each trajectory (or episode) **do**
5: Compute Advantage:

$$A(s_t, a_t) = r_{t+1} + \gamma V(s_{t+1}; w) * (1 - d) - V(s_t; w) \quad (7.12)$$

▷ $A(s_t, a_t) \approx \delta_t$, TD error

6: Update Critic:

$$w \leftarrow w + \alpha_w \nabla_w V(s_t; w) A(s, a) \quad (7.13)$$

▷ Critic minimises TD error

7: Update Actor:

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi(a|s; \theta) A(s, a) + \beta H(\pi_\theta(s_t)) \quad (7.14)$$

▷ maximises actor loss function

8: **end for**
9: **end for**

Sample efficiency: Use a replay memory with a set of (s, a, r, s') tuples and sample a mini-batch for update.

Note: d is a done indicator for the episode end

Advantage provides a better signal by focusing on relative action quality. Could use multi-step bootstrapping for more stability and faster reward propagation.

Adding the **entropy** encourages the policy to stay uncertain/spread out and explore.

Source: https://medium.com/@kswagat1979/advantage-actor-critic-a2c-models-ab188ddfcb72

Natural Policy Gradients (NPG)

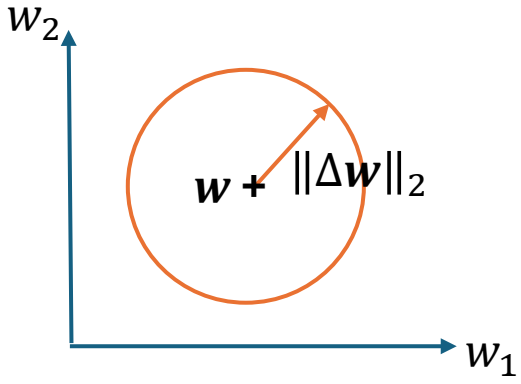
Notation: θ and w are often used interchangeably!

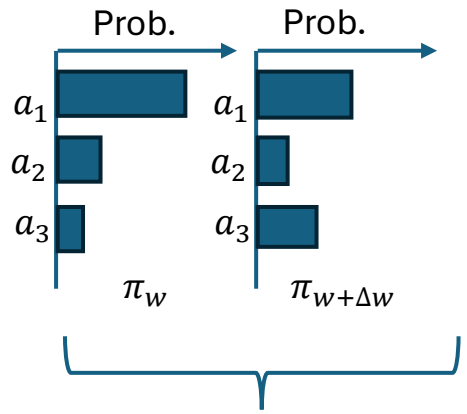
- Regular policy gradient methods
  - Update policy parameters $\boldsymbol{w}$ in the direction of the steepest ascent of the performance measure $J(\boldsymbol{w}) \stackrel{\text{def}}{=} v_{\pi_w}(s_0)$
  - The update changes the parameters $\Delta \boldsymbol{w} \propto \nabla_w J(\boldsymbol{w})$. The step size α restricts **moves in the parameter space** to some $\|\Delta \boldsymbol{w}\|_2$.
- **Issue**: The same amount of change in the parameter space $\boldsymbol{w}$ can have very different effects on the change of the stochastic policy π_w (the probability distribution over actions). This can cause **instability**.
- **Idea**: Natural policy gradients take the geometry of the policy (a probability manifold) into account.
  - **Measure of the difference between two policies**' action probability distributions**:** Kullback-Leibler (KL) divergence $D_{KL}(\pi_w || \pi_{w+\Delta w})$
  - Constraint $\Delta \boldsymbol{w}$ by a maximum allowable policy change using D_{KL}.
  - Local approximation: The Fisher information criterion can be used to **adjust the gradient** for changes in policy distribution. α is now a constraint on how much the policy changes:

$$\Delta \boldsymbol{w} \propto F_w^{-1} \nabla_w J(\boldsymbol{w})$$

- **Problem**: Calculating the Fisher information matrix (F_w) is too expensive for deep learning models. We need an approximation for the approximation!

  This idea inspired several state-of-the-art methods:
  - Trust Region Optimization (TRPO)
  - Proximal Policy optimization (PPO)



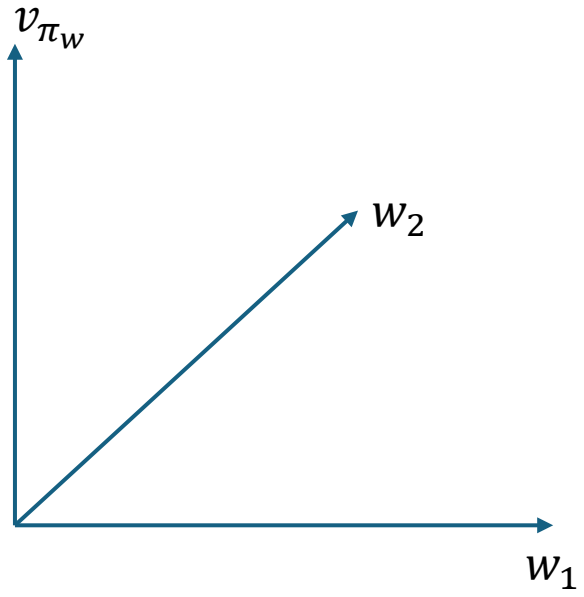


$D_{KL}(\pi_w || \pi_{w+\Delta w})$

Notation: θ and w are often used interchangeably!

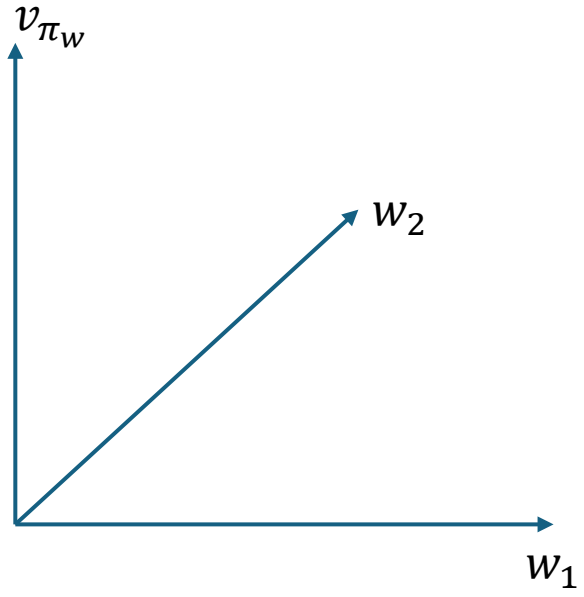
Standard vs. Natural Policy Gradient

Standard Gradient

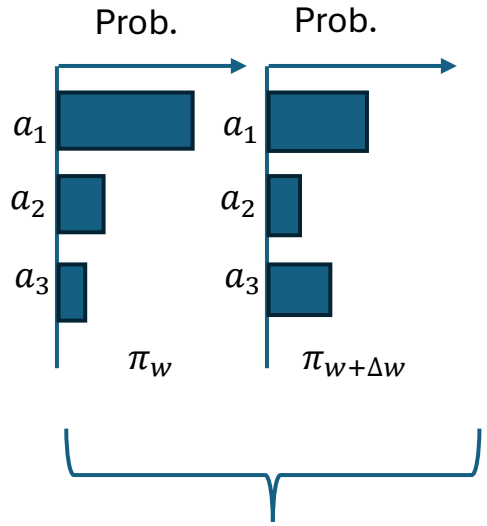


Moving a fixed distance in the **parameter space**. This can lead to large policy changes.

Natural Policy Gradient



Moving a fixed distance in the **policy space**.



$D_{KL}(\pi_w||\pi_{w+\Delta w})$

Trust Region Policy Optimization (TRPO; Schulman et al, 2015)

Notation: θ and w are often used interchangeably!

- Simplification of Natural Policy Gradient.
- Surrogate objective. Instead of directly maximizing the value function, it finds the update that increases the probability of picking high-advantage actions the most.

$$\max_{\Delta w} \mathbb{E}_{\pi_w}\left[\frac{\pi_{w+\Delta w}(s,a)}{\pi_w(s,a)} A^{\pi_w}(s,a)\right]$$

- Under a constrain that explicitly bound of the policy change per update step.

$$\mathbb{E}\left[D_{KL}(\pi_w(s,\cdot)||\pi_{w+\Delta w}(s,\cdot))\right] \leq \delta$$

- δ is a tune-able hyperparameter specifying the size of the safe "trust region." This prevents destructive large changes in the policy.

Proximal Policy Optimization (PPO; Schulman et al, 2017)

Notation: θ and w are often used interchangeably!

- A popular and more practical variant of TRPO.
- Replaces expensive KL-divergence constraint with a clipped surrogate objective function to penalize overly large policy changes with at probability ratio outside of $[1-\epsilon, 1+\epsilon]$

$$\max_{\Delta w} \mathbb{E}_{\pi_w}\left[\min\left(\frac{\pi_{w+\Delta w}(s,a)}{\pi_w(s,a)}(w)A^{\pi_w}(s,a), \text{clip}\left(\frac{\pi_{w+\Delta w}(s,a)}{\pi_w(s,a)}1-\epsilon, 1+\epsilon\right)A^{\pi_w}(s,a)\right)\right]$$

- ϵ is a tunable hyperparameter.
- Achieves much of the stability of NPG methods without the computational cost.

What you Should Know

- DRL uses neural networks for non-linear value function or policy approximation resulting in:
  - more powerful for high-dimensional and continuous state and action spaces, and
  - does not need manual state feature engineering.
  - However, most convergence guarantees are gone (for semi-gradient methods, for linear approximation, etc).
- Instability from the “Deadly Triad” (e.g., in DQN)
  - Function approximation
  - Bootstrapping (moving targets using current estimates)
  - Off-policy learning (learning from data generated by a different policy)
- Lead to
  - Non-linear approximation means that small errors can get amplified through bootstrapping
  - The network chases a moving target.
  - Training can diverge or oscillate wildly.
- Many tricks are needed
  - Experience replay → decorrelate data, improve sample efficiency, requires off-policy learning
  - DQN uses slower-moving target networks → reduce instability due to moving targets
  - Double DQN → decouple next action selection and q-value estimation to reduce overestimation bias
  - Advantage normalization → reduce variance
  - Natural policy gradients → improve stability